Dijkstra's Algorithm

Algorithm Design

INTRODUCTION

Dijkstra's algorithm

Dijkstra's algorithm allows us to find the shortest path between any two vertices of a graph.

- **It is the solution to the single-source shortest path tree problem in graph theory.**
- **It differs from the minimum spanning tree because the shortest distance between two vertices might not include all the vertices of the graph.**
- **Works on both directed and undirected graphs. However, all edges must have nonnegative weights.**

Edsger Wybe Dijkstra was a Dutch computer scientist, programmer, software engineer, systems scientist, science essayist, and pioneer in computing science.

Weighted graph *is a* ***graph*** *in which each branch is given a numerical* ***weight****. (which are usually taken to be positive).*

HOW IT WORKS?

Approach: Greedy (To find the single source shortest path)

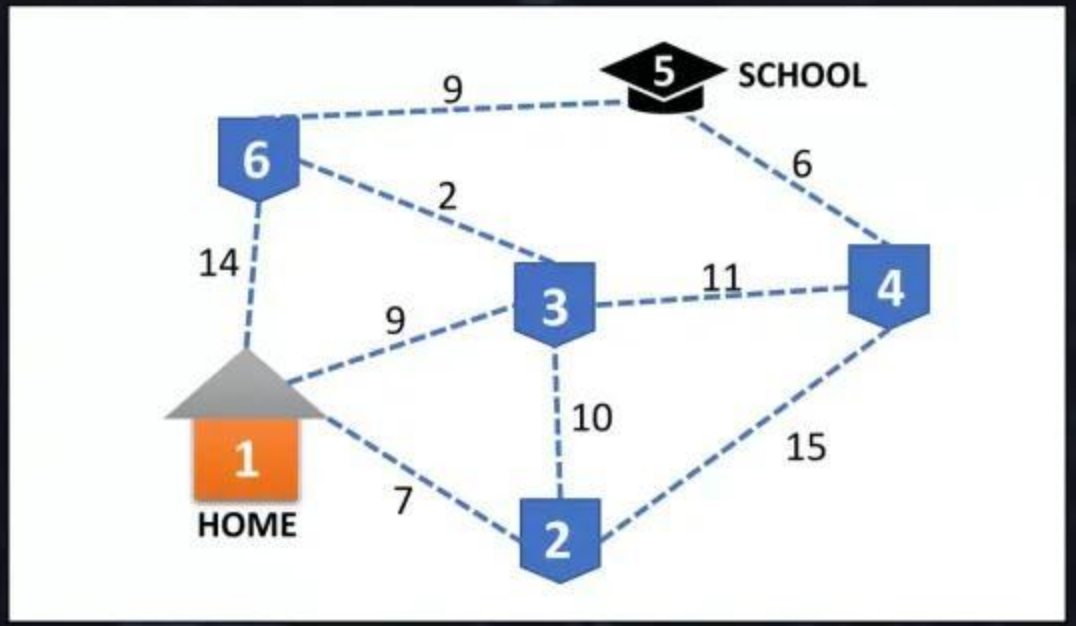
- Input: Weighted graph G={E,V} and source vertex v∈V, such that all edge weights are nonnegative.
- Output: Lengths of shortest paths (or the shortest paths themselves) from a given source to all other vertices.

Example:

A student wants to go from home to school in the shortest possible way. She knows some roads are heavily congested and difficult to use. In Dijkstra's algorithm, this means the edge has a large weight/cost and the shortest path tree found by the algorithm will try to avoid edges with larger weights. If the student looks up directions using a map service, it is likely they may use Dijkstra's algorithm.

For Example: The shortest path, which could be found using Dijkstra's algorithm, is

1(HOME) → 3 → 6 → 5(SCHOOL)



IMPLEMENTATION

Suppose,
Distance be v, Source be u & Cost be c

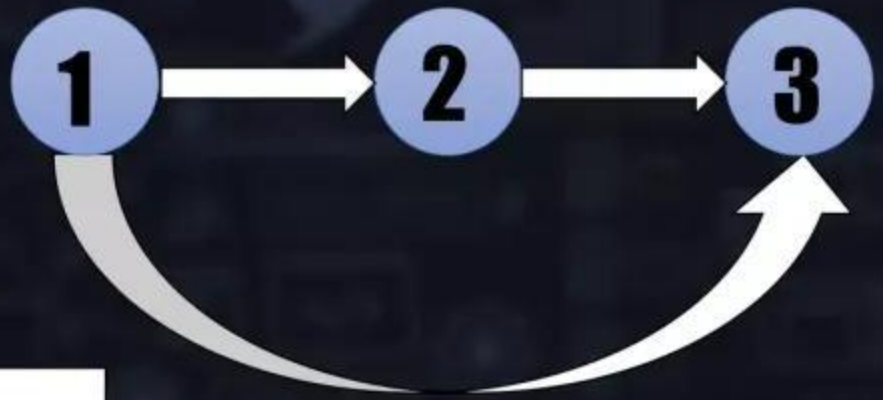
So we implement the greedy method in two steps:

Step 1:

$$d(u) + c(u,v) < d(v)$$

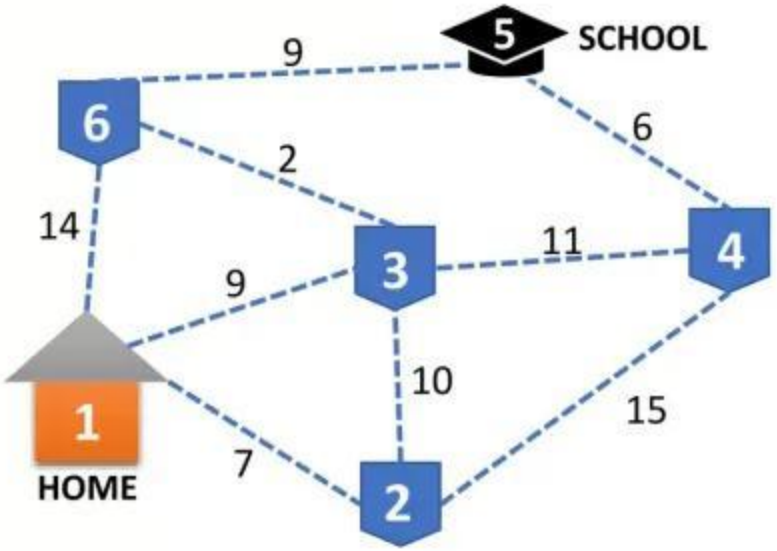
Step 2:

$$d(v) = d(u) + c(u,v)$$



Single Source Shortest Path - Greedy Method

- *Given a graph and a source vertex in the graph, find shortest paths from source to all vertices in the given graph.*



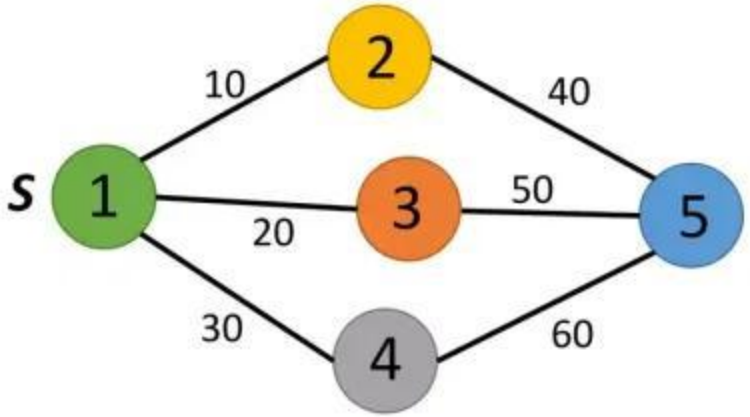
Source	Destination

Final Shortest Path:

PSEUDO CODE:

PSEUDO CODE:

```
Dijkstra(Graph, Source)
Create Vertex Set Q
      for each vertex V in graph
            distance[V] = infinite
            previous[V] = NULL
            If V not equal S,
              add V to Priority Queue Q
       distance[S] = 0

while Q IS NOT EMPTY
      U = Extract - min [Q]
      for each neighbour V of U
            relax(U, V)
```

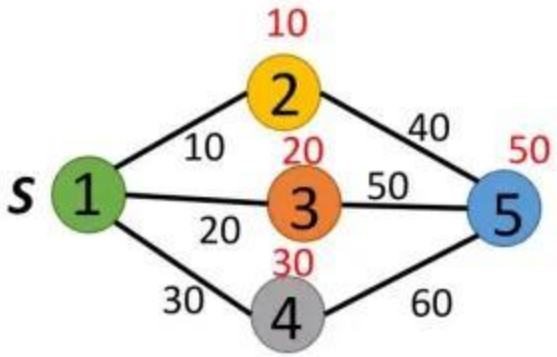



```
function dijkstra(G, S)
      for each vertex V in G
            distance[V] <- infinite
            previous[V] <- NULL
            If V != S, add V to Priority Queue Q
      distance[S] <- 0

while Q IS NOT EMPTY
      U <- Extract MIN from Q
      for each unvisited neighbour V of U
            tempDistance <- distance[U] + edge_weight(U, V)
            if tempDistance < distance[V]
                  distance[V] <- tempDistance
                  previous[V] <- U
return distance[], previous[]
```

TIME COMPLEXITY

PSEUDO CODE: (From previous slide)

```
Dijkstra(Graph, Source)
Create Vertex Set Q
    for each vertex V in graph
        distance[V] = infinite
        previous[V] = NULL
        If V not equal S,
            add V to Priority Queue Q
    distance[S] = 0

while Q IS NOT EMPTY
    U = Extract - min [Q]
    for each neighbour V of U
        relax(U, V)
```



TIME COMPLEXITY

Dijkstra's Algorithm Complexity:

- Time Complexity: **O(E Log V)**

where, E is the number of edges and V is the number of vertices.

- Space Complexity: **O(V)**

Time Complexity:
It is the computational complexity that describes the amount of computer time it takes to run an algorithm.

Pseudocode:
It is the plain language description of the steps in an algorithm or another system.

Space complexity:
It is the amount of memory used by the algorithm (including the input values to the algorithm) to execute and produce the result

ADVANTAGES & DISADVANTAGES:

Advantages:

1. It is used in Google Maps
2. It is used in finding Shortest Path on geographical Maps.
3. To find locations of Map which refers to vertices of graph. Distance between the location refers to edges.
4. It is used in IP routing to find Open shortest Path First.
5. It is used in the telephone network.

Disadvantages:

1. The major **disadvantage** of the **algorithm** is the fact that it does a blind search there by consuming a lot of time waste of necessary resources
2. It cannot handle negative edges.
3. This leads to acyclic graphs and most often cannot obtain the right shortest path.
4. Time consuming because it expands in every direction

APPLICATIONS:

Dijkstra's Algorithm has several real-world use cases, some of which are as follows:

Digital Mapping Services in Google Maps:

Dijkstra's Algorithm is used to find the minimum distance between two locations along the path.

Social Networking Applications:

The standard Dijkstra algorithm can be applied using the shortest path between users measured through handshakes or connections among them.

IP routing to find Open shortest Path First: Open Shortest Path First (OSPF) :

The algorithm provides the shortest cost path from the source router to other routers in the network.

Robotic Path:

Robot/drone moves in the ordered direction by following the shortest path to keep delivering the package in a minimum amount of time.